
Facial AI — Spatial Memory Change Detection for Visually Challenged Users

An edge-AI mobile system that creates personalized spatial memory of frequently visited places and alerts users when something important has changed.

One-sentence pitch: *"Our system remembers places for visually challenged users and tells them when something important has changed."*

The Core Idea

A normal obstacle-detection app says: *"There is a chair."*

Our system goes one step further: *"There wasn't a chair here during your previous visit, and now it is blocking your usual path."*

That is **personalized spatial change detection** — not just *"What is around me?"* but *"What has changed since I was last here?"*

Prototype Strategy

The prototype focuses on **proving the main idea clearly** using one corridor/room as the test environment. We don't need to build the full AI system — just the 3 core features.

3 Core Features

Feature 1 — SAVE (Create Spatial Memory)

Camera → AI → Detect Objects → Save to Database

User opens the app, taps **SAVE THIS PLACE**, camera captures the environment, AI detects objects (Door, Chair, Stairs), and the app saves the spatial memory.

Feature 2 — CHECK (Detect Changes)

Camera → AI → Detect Current Objects → Compare with Saved Memory

When the user returns, they tap **CHECK FOR CHANGES**. The camera captures the current environment, AI detects objects, and the system compares old vs. new.

Feature 3 — ALERT (Warn About Important Changes)

Important Change Detected → Risk Score → Voice Alert + Vibration

If something important changed (chair blocking path), the system speaks a warning. If something minor changed (poster moved), it stays silent.

Prototype Demo Walkthrough

This is how the actual demo/presentation should work:

Demo 1 — Save the environment (Normal)

Show a corridor or room to the camera.

Press: **SAVE PLACE**

Output: **"Environment saved. MCA Corridor memory created."**

The AI detects and stores: Door, Chair (on the side), Stairs.

Demo 2 — Introduce a change

Physically put a **chair in the walkway**.

Before:

Entrance → Empty Corridor → Classroom

After:

Entrance → CHAIR IN WALKWAY → Classroom

Press: **CHECK CHANGES**

Output: **"Change detected."**

Demo 3 — Important change alert

The system announces:

"Warning. A chair has appeared in your usual path."

The phone vibrates.

Risk level displayed: **HIGH**

Demo 4 — Minor change (no alert)

Move a **poster** on the wall slightly.

Press: **CHECK CHANGES**

Output: **"No important change detected."**

This is critical because it shows the **innovation** — the system doesn't alert for every change, only **important** ones.

Architecture Overview

[Diagram — see the markdown document for interactive view]

Prototype Pipeline (Simplified)

```
Phone Camera
  |
  v
YOL011
  |
  v
Object Detection (Chair 92%, Door 96%, Stairs 88%)
  |
  v
Spatial Memory (MongoDB)
  |
  v
Compare OLD vs NEW
  |
```



System Components

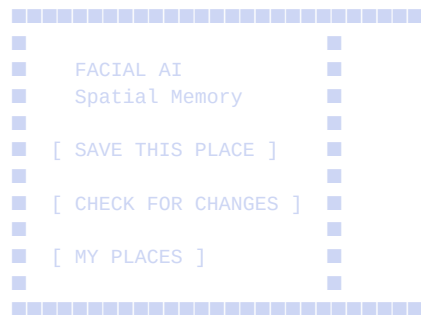
The system is split into **two main parts**:

Component	Technology	Built In
Android App	Kotlin, CameraX, Retrofit, TTS	Android Studio
AI Backend	Python, FastAPI, YOLOv8/YOLO11, Gemini, MongoDB	Antigravity IDE

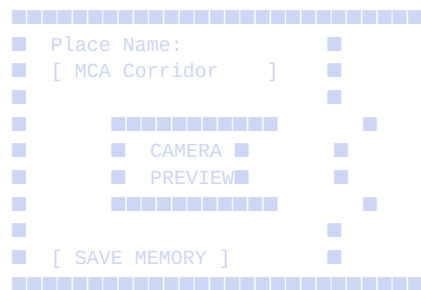
Android App Screens

The prototype needs only **4 screens**:

Screen 1 — Home



Screen 2 — Save Place



Screen 3 — Check Changes





Screen 4 — Alert



Text-to-Speech reads the alert aloud automatically.

Proposed Changes — Technical Details

Component 1: Python AI Backend (Built in Antigravity IDE)

This is the brain of the system. It receives images, detects objects, stores spatial memory, and detects changes.

[NEW] `backend/main.py`

FastAPI server with the following endpoints:

Endpoint	Method	Purpose
<code>POST /api/save-place</code>	POST	Save a new place's spatial memory
<code>POST /api/check-changes</code>	POST	Compare current view with stored memory
<code>GET /api/places</code>	GET	List all saved places
<code>GET /api/place/{id}</code>	GET	Get details of a specific place
<code>DELETE /api/place/{id}</code>	DELETE	Delete a place's memory

[NEW] `backend/detection/yolo_detector.py`

Local object detection using YOLOv8/YOLO11 (Ultralytics):

What it does:

- Receives an image
- Runs YOLO to detect objects
- Returns a list of detected objects with:
- Object name (chair, car, person, etc.)


```

    "_id": "ObjectId",
    "name": "MCA Corridor",
    "user_id": "default",
    "created_at": "2026-08-15T22:00:00Z",
    "updated_at": "2026-08-15T22:00:00Z",
    "scene_description": "A corridor with a door on the left and stairs at the end.",
    "reference_image_path": "/images/mca_corridor.jpg",
    "objects": [
      {
        "name": "chair",
        "x_center": 0.32,
        "y_center": 0.65,
        "width": 0.15,
        "height": 0.20,
        "confidence": 0.89,
        "danger_score": 7,
        "spatial_zone": "middle-left"
      },
      {
        "name": "door",
        "x_center": 0.52,
        "y_center": 0.20,
        "width": 0.18,
        "height": 0.40,
        "confidence": 0.95,
        "danger_score": 8,
        "spatial_zone": "top-center"
      }
    ]
  }
}

```

Collection: `change_log`

```

{
  "_id": "ObjectId",
  "place_id": "ObjectId (ref to places)",
  "change_type": "new_object",
  "object_name": "barricade",
  "description": "A barricade has appeared in your usual path.",
  "risk_level": "high",
  "danger_score": 10,
  "detected_at": "2026-08-16T10:30:00Z"
}

```

TIP: Why MongoDB? Objects are naturally embedded inside places (no JOINS needed), schema is flexible for adding new fields, and document structure mirrors the JSON the API already uses.

[NEW] `backend/comparison/change_detector.py`

The comparison engine — the **core innovation** of the project:

What it compares:

Check	Description	Example
New objects	Object exists now but didn't before	Barricade appeared
Removed objects	Object existed before but is gone	Stairs removed
Moved objects	Same object, different position	Chair moved to center
Scene changes	Overall scene differs significantly	Entrance changed

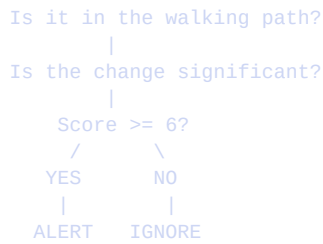
Risk Analysis Logic:

The system checks 3 things for every change:

```

Is the object important?
|

```



Danger scoring system:

```
DANGER_SCORES = {
    # Navigation blockers (Critical)
    'barricade': 10, 'construction': 10,
    'stairs': 10, 'fire hydrant': 9,

    # Path obstacles (High)
    'car': 9, 'truck': 9, 'bus': 9, 'motorcycle': 8,
    'bicycle': 8, 'chair': 7, 'bench': 7,
    'dining table': 7, 'suitcase': 7,

    # Moderate obstacles
    'potted plant': 5, 'backpack': 5,
    'umbrella': 4, 'box': 5,

    # Low importance
    'book': 2, 'cup': 1, 'bottle': 2,
    'cell phone': 1, 'laptop': 2,

    # People (usually temporary)
    'person': 1, 'dog': 3, 'cat': 2,
}

ALERT_THRESHOLD = 6 # Only alert for score >= 6
```

Example 1 — Barricade detected:

Barricade detected → Danger score = 10 → 10 >= 6 → ALERT

Example 2 — Poster changed:

Poster changed → Danger score = 1 → 1 < 6 → IGNORE

Position-based comparison:

The system stores object positions (as % of image). When the same object is detected but its position shifted by more than **15%**, it's flagged as "moved".

```
OLD: Chair → x=20%, y=60% (left side)
NEW: Chair → x=50%, y=60% (center of walkway)

Difference: 30% shift → LARGE
Object: Chair → Danger score 7
Result: ALERT — "A chair has moved into your usual path."
```

[NEW] backend/requirements.txt

```
fastapi==0.115.0
uvicorn==0.30.0
ultralytics==8.2.0
google-generativeai==0.8.0
Pillow==10.4.0
python-multipart==0.0.9
opencv-python-headless==4.10.0.84
numpy==1.26.4
pydantic==2.9.0
pymongo==4.8.0
motor==3.5.0
```

`pymongo` is the MongoDB Python driver. `motor` is the async version for FastAPI.

Component 2: Android App (Built in Android Studio)

IMPORTANT: The Android app must be built in **Android Studio**. I will generate all the code files here, and you'll create the Android Studio project and paste/copy the files in.

[NEW] Android Project Structure

```
app/
├── src/main/
│   ├── java/com/facialai/
│   │   ├── MainActivity.kt           # Home screen
│   │   ├── SavePlaceActivity.kt     # Save new place
│   │   ├── CheckChangesActivity.kt  # Check for changes
│   │   ├── MyPlacesActivity.kt      # View saved places
│   │   ├── AlertActivity.kt         # Show/speak alerts
│   │   └── api/
│   │       ├── ApiService.kt        # Retrofit API interface
│   │       └── RetrofitClient.kt    # HTTP client setup
│   │       └── models/
│   │           ├── Place.kt          # Place data model
│   │           ├── DetectedObject.kt # Object data model
│   │           └── ChangeAlert.kt     # Alert data model
│   │       └── utils/
│   │           ├── CameraHelper.kt   # CameraX setup
│   │           ├── SpeechHelper.kt   # Text-to-Speech
│   │           └── VibrationHelper.kt # Vibration patterns
│   └── res/
│       ├── layout/
│       │   ├── activity_main.xml
│       │   ├── activity_save_place.xml
│       │   ├── activity_check_changes.xml
│       │   ├── activity_my_places.xml
│       │   └── activity_alert.xml
│       ├── values/
│       │   ├── colors.xml
│       │   ├── strings.xml
│       │   └── themes.xml
│       └── drawable/
│           └── (icons and assets)
├── AndroidManifest.xml
├── build.gradle.kts
└── ...
```

Key Android Dependencies

```
// CameraX for camera
implementation("androidx.camera:camera-core:1.3.4")
implementation("androidx.camera:camera-camera2:1.3.4")
implementation("androidx.camera:camera-lifecycle:1.3.4")
implementation("androidx.camera:camera-view:1.3.4")

// Retrofit for HTTP
implementation("com.squareup.retrofit2:retrofit:2.11.0")
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
implementation("com.squareup.okhttp3:okhttp:4.12.0")

// Material Design
implementation("com.google.android.material:material:1.12.0")
```

How It All Connects

[Diagram — see the markdown document for interactive view]

Edge AI — Why It Matters

The problem statement mentions **mobile/edge AI**. This means the AI runs directly on or near the phone, not in a distant cloud.

CLOUD APPROACH: Phone → Internet → Server → AI → Result (slow, needs internet)
OUR APPROACH: Phone → Local AI Server → Result (fast, works offline)

Advantages of edge AI (strong point for judges):

- Faster response time
- Works with poor/no internet
- Better privacy (images stay local)
- Less data sent to servers

Build Order (Step-by-Step)

Phase 0: Setup (Prerequisites)

Step	What	Time
0.1	Install MongoDB Community Edition	15 min
0.2	Install Android Studio (if not installed)	30 min
0.3	Get Gemini API key from Google AI Studio	5 min
0.4	Verify Python 3.10+ is installed	5 min

MongoDB Installation (Windows)

- Download **MongoDB Community Server** from mongodb.com/try/download/community
- Choose **Windows x64, MSI** package
- Run the installer → choose **Complete** installation
- Check **"Install MongoDB as a Service"** (runs automatically)
- Check **"Install MongoDB Compass"** (GUI tool to view your data)
- After install, verify by opening PowerShell:

mongosh

You should see the MongoDB shell connect. Type **exit** to quit.

Phase 1: Android App (In Android Studio) — We build this FIRST

Step	What	Time
1.1	Create Android Studio project "Facial AI"	10 min

Step	What	Time
1.2	Build Home screen UI	20 min
1.3	Implement CameraX integration	30 min
1.4	Build Save Place screen	30 min
1.5	Build Check Changes screen	30 min
1.6	Build Alert screen + Text-to-Speech	20 min
1.7	Build My Places screen	20 min
1.8	Add vibration patterns	10 min
1.9	Add Retrofit API client (connects to backend)	20 min

Phase 2: AI Backend (In Antigravity IDE)

Step	What	Time
2.1	Set up FastAPI project structure	15 min
2.2	Set up MongoDB connection + collections	20 min
2.3	Implement YOLO object detection	30 min
2.4	Implement Gemini scene analyzer	30 min
2.5	Build spatial memory (MongoDB CRUD)	30 min
2.6	Build comparison/change detection engine	45 min
2.7	Wire up API endpoints	30 min
2.8	Test with sample images	20 min

Phase 3: Integration and Polish

Step	What	Time
3.1	Connect Android app to backend	30 min
3.2	End-to-end testing	30 min
3.3	UI polish and accessibility	30 min
3.4	Demo preparation	20 min

Build It in 3 Stages

Stage 1 — Basic Demo (Get it working)

Get: Camera → Object Detection → Object names → Voice output
Example output: **"Chair detected. Door detected. Stairs detected."**

Stage 2 — Spatial Memory (Remember places)

Add: SAVE MEMORY → Objects + positions → Database
Then: CHECK → New objects + positions → Compare

Stage 3 — Intelligent Change Detection (The innovation)

Add: Change → Importance Score → Risk → Personalized Alert
This is where the project becomes impressive.

Team Roles (6 Members)

Member	Role	Responsibility
M1	Android + Camera	CameraX setup, UI screens, navigation
M2	AI / YOLO11	Object detection, model integration
M3	Memory / Database	MongoDB setup, save/retrieve spatial memory
M4	Comparison Engine	Image/position comparison, change detection
M5	Risk + Alerts	Danger scoring, Voice (TTS), Vibration
M6	Integration + Testing	GitHub, build pipeline, end-to-end testing

Pipeline:

M1 (Camera) → M2 (YOLO) → M3 (Memory) → M4 (Compare) → M5 (Alert) → M6 (Test)

Prototype vs Final Product

Feature	Prototype	Final Version
Locations	One location	Multiple locations
Objects	Few objects (COCO 80)	Many object classes (custom)
Comparison	Image + position comparison	Spatial/3D memory
AI Model	YOLOv8n / YOLO11n	Custom optimized AI
Depth	Basic position	Depth sensing + SLAM
Risk	Simple danger score	Personalized risk model
Alerts	Voice + vibration	Voice + vibration + wearable

Feature	Prototype	Final Version
Camera	Phone camera	Phone + smart glasses
Database	Local MongoDB	Advanced personalized memory
Scope	Prove the concept	Production-ready system

Presentation Tips

IMPORTANT: Don't just show screenshots. Physically demonstrate the prototype. Have a chair in a corridor, save the environment, move the chair, scan again, and let the phone speak the warning. That will make the concept immediately understandable to judges—even before you explain the AI.

If a judge asks "What exactly are you building?", say:

"We are developing an edge-AI mobile system that learns a visually challenged user's familiar environment and compares future visits with that spatial memory to detect meaningful environmental changes and provide timely audio or vibration alerts."

What makes this different from existing solutions:

Existing apps say: "There is a chair."

Our system says: "There wasn't a chair here during your previous visit, and now it is blocking your usual path."

One important note for presentations:

Don't claim: "Our AI perfectly understands the entire environment."

Instead say: "Our prototype focuses on detecting meaningful changes in frequently visited environments, particularly objects and obstacles that may affect the user's familiar path."

That is realistic and technically defensible.

User Review Required

IMPORTANT: MongoDB: Install MongoDB Community Edition (see Phase 0 steps above).

IMPORTANT: Gemini API Key: You'll need a free Google Gemini API key from [Google AI Studio](#). Please get one before we start Phase 2.

IMPORTANT: Python Required: The backend needs Python 3.10+. Please confirm you have Python installed (run `python --version` in your terminal).

IMPORTANT: Network Setup: When testing with a physical Android device, the phone and your PC must be on the **same WiFi network** so the app can reach the Python backend server.

Decisions Made

Decision	Choice
App name	Facial AI
Database	MongoDB

Decision	Choice
Build order	Android app first, then backend
Language	Kotlin
AI stack	YOLOv8/YOLO11 (local) + Gemini (cloud)
Prototype scope	3 features: SAVE, CHECK, ALERT

Verification Plan

Automated Tests

- Unit tests for the comparison engine (`pytest`)
- API endpoint tests with sample images
- Object detection accuracy tests

Manual Verification (Demo Scenarios)

- **Demo 1 (SAVE):** Save a corridor → Output: "Environment saved."
- **Demo 2 (CHANGE):** Add a chair in walkway → Output: "Change detected."
- **Demo 3 (ALERT):** Chair blocking path → Voice: "Warning. A chair has appeared in your usual path." + Vibration
- **Demo 4 (IGNORE):** Move a poster → Output: "No important change detected."
- **Demo 5:** Full end-to-end on physical Android phone with voice alert